TP GAMARRA Damien

Présentation :

Le but du TP est de prévoir la base de données d'une version de Discord. On part de 5 000 utilisateurs, 150 serveurs, environ 970 salons, 300 000 messages et 15 000 amis.
Discord devait stocker de plus en plus de messages tout en gardant des temps de réponse rapides. [A1]
En 2015, MongoDB a commencé à rencontrer des problèmes de performance vers 100 millions de messages stockés. [A1]
Discord est passé à Cassandra pour mieux répartir les données, mais les partitions et les suppressions ont posé problème. [A1]
En 2022, Discord a migré vers ScyllaDB pour améliorer les performances et réduire le nombre de machines nécessaires. [A2]
Voici pourquoi il faut choisir une base de donnée adapté à son volume et a sa manière dont les messages sont utilisés

Chiffre	A quoi ça sert ?
100 millions de messages par jour en 2016 [A1]	Calculer les messages par seconde
100 millions de messages stockés en 2015 [A1]	Comprendre le manque de RAM
100 Mo par partition Cassandra [A1]	Comparer les façons de découper les messages

1.

1.a : Utilisateurs, données et définitions
Un utilisateur rejoint des serveurs, discute dans des salons et ajoute des amis. Dans un serveur, son rôle est membre ou modérateur ou propriétaire. On doit aussi savoir qui est en ligne et compter les messages de chaque jour.

Donnée	Création ou changement	Fin
Utilisateur	Inscription avec pseudo et e-mail	Suppression à préciser
Serveur	Créé avec un propriétaire	Suppression à préciser
Appartenance	Un utilisateur rejoint un serveur et reçoit un rôle	Départ à confirmer [P]
Salon	Créé dans un serveur, en texte ou vocal	Suppression à préciser
Message	Envoyé dans un salon texte, parfois modifié	Suppression selon les droits à définir
Pièce jointe	Image, fichier ou lien sur un message	Sort à préciser [P]

Amitié	Deux utilisateurs deviennent amis	Retrait possible [P]
Présence	Mise à jour quand la personne est active	Expire après 5 minutes
Statistique	Calculée par serveur et par jour	Recalcul possible [P]

Glossaire :

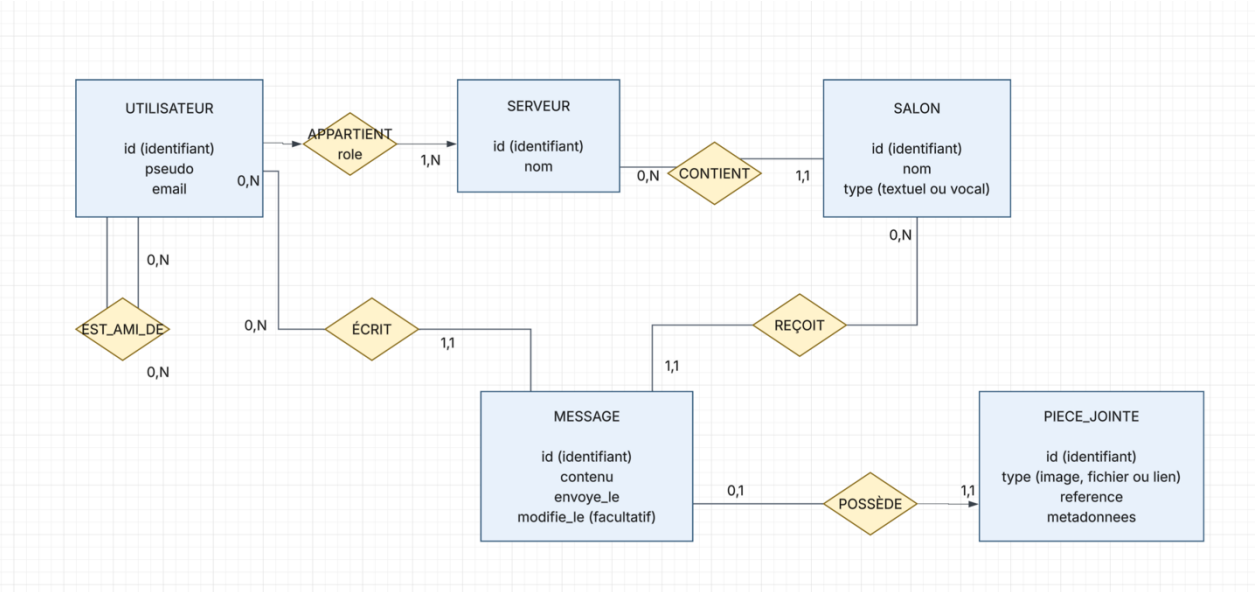
Mot	Ce que ça veut dire
Utilisateur	Personne qui a un compte
Serveur	Groupe Discord
Salon	Endroit où on écrit ou parle
Membre	Personne dans un serveur
Rôle	Membre, modérateur ou propriétaire
Propriétaire	Personne qui possède le serveur
Message	Texte envoyé dans un salon
Pièce jointe	Image, fichier ou lien dans un message
Amitié	Lien entre deux comptes, dans les deux sens
Présence	Indique si une personne est en ligne

1.b : Les règles à respecter

Dans la base	Dans le code	À part
Un e-mail unique	Vérifier l'inscription [P]	
Une appartenance par utilisateur et serveur	Garder un propriétaire dans le serveur	
Rôle limité à trois valeurs, un seul propriétaire	Gérer le changement de propriétaire	
Nom de salon unique dans son serveur		
Un message a un auteur et un salon	Vérifier le salon texte et l'appartenance	
Une pièce jointe au maximum par message	Contrôler son type [P]	
Pas de doublon d'amitié ni d'amitié avec soi-même	Ranger les deux ID dans le même ordre	
Une statistique par serveur et par jour	Compter les messages chaque jour	
	Calculer les suggestions	Présence dans Redis, durée de 300 s [P]

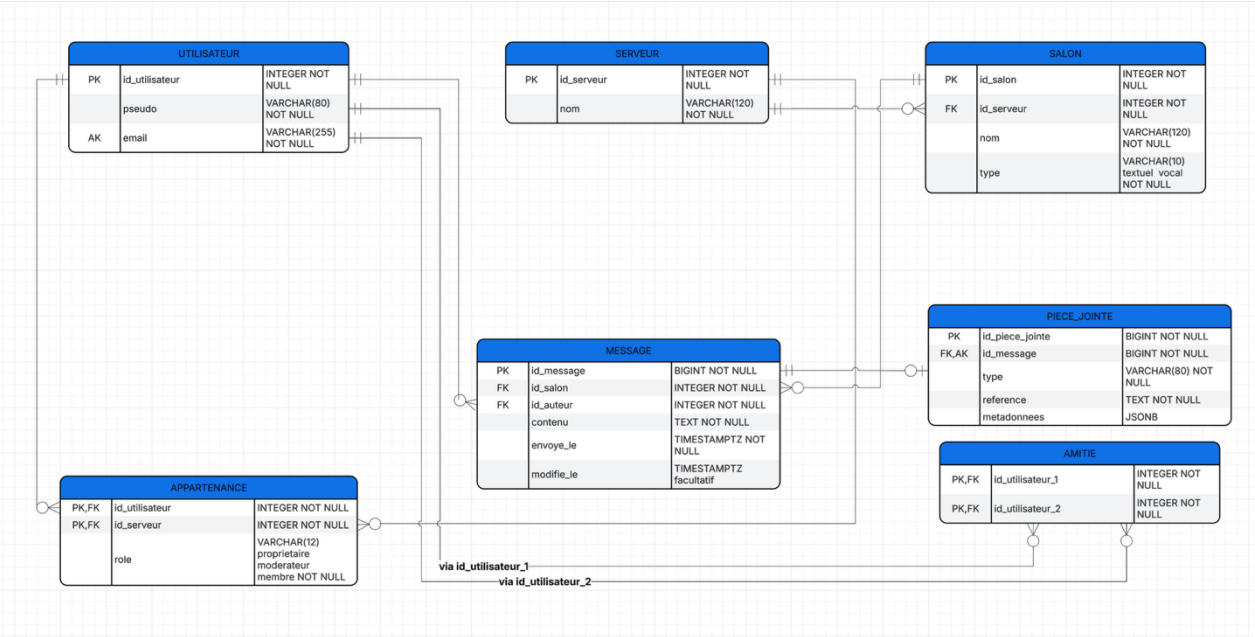
La base peut empêcher d'avoir deux propriétaires. Pour éviter d'en avoir zéro, il faut aussi vérifier les départs et les transferts.

1.c : Mon MCD



Question du TP	Mon choix et ce que j'écarte
Rôle	Un champ role dans l'appartenance, pas une table de rôles
Amitié	Un seul lien par paire, pas A-B et B-A
Propriétaire	Un membre avec le rôle propriétaire, pas un second champ dans serveur
Pièce jointe	Une entité à part, pas des champs dans message
Message	Une entité du MCD, stockée dans PostgreSQL pour ce TP [P]

1.d : Passage au MLD et au MPD



Pour passer au MLD, une relation 1,N donne une clé étrangère du côté N. Une relation N,N devient une table de liaison. L'amitié utilise deux liens vers la table utilisateur. Le MPD PostgreSQL et ses contraintes SQL sont en annexe.

PK = clé primaire / FK = clé étrangère / UQ = valeur unique / ? = facultatif.

Index prévu	À quoi il sert
message(salon_id, id DESC)	Lire les 50 derniers messages
message(salon_id, envoye_le)	Compter les messages d'un jour
appartenance(utilisateur_id, serveur_id)	Trouver les serveurs d'un compte
amitie(utilisateur_max, utilisateur_min)	Chercher les amis dans les deux sens
UNIQUE(serveur_id, nom)	Trouver un salon sans doublon

Le code doit aussi refuser un message dans un salon vocal ou venant d'un non-membre. Pour trier par id DESC, je prévois des ID de message qui suivent l'ordre du temps [P].

1.e : Cinq questions avant de dev

Question	Ce que ça change
Si le propriétaire quitte son serveur, que fait-on ?	Transfert du rôle ou départ refusé
Que supprime-t-on avec un compte ou un serveur ?	Règles de suppression
Une pièce jointe contient le fichier ou juste son lien ?	Stockage des fichiers
Qui peut modifier ou supprimer un message ?	Droits à prévoir

Faut-il bloquer des suggestions entre certains comptes ?	Filtres à ajouter

2. Ensuite, regarder les accès et les volumes

2.a : Ce que l'application fait le plus
On doit lire les 50 derniers messages d'un salon est l'action la plus fréquente. Pour les autres fréquences et les temps visés, ce sont mes estimations [P]. Les 80 ms viennent du seuil suivi par Discord en 2017 [A1].

Action	L/E	Fréquence	Temps visé [P]	Forme	Cohérence
Lire 50 messages	L	N° 1	p95 < 80 ms	Plage triée	Petit retard possible
Envoyer un message	E	Forte [P]	p95 < 80 ms	Clé	Stricte
Vérifier le droit d'écrire	L	Chaque envoi	p95 < 80 ms	Relations	Stricte
Signaler sa présence	E	Régulière [P]	p95 < 80 ms	Clé + durée	Temporaire
Voir les personnes en ligne	L	Forte [P]	p95 < 80 ms	Clés	Temporaire
Ajouter un ami	E	Faible [P]	p95 < 80 ms	Paire	Stricte
Suggérer des amis	L	Moyenne [P]	p95 < 200 ms	Relations	Retard possible
Voir les statistiques	L	Selon usage [P]	p95 < 200 ms	Calcul par jour	Quotidienne
Mettre à jour les statistiques	L/E	Chaque jour	< 24 h	Calcul + écriture	Quotidienne
Supprimer un message	E	Faible [P]	p95 < 80 ms	ID	Stricte

2.b : Les calculs demandés
Messages de décembre 2016 [A1, H2]
Moyenne : 100 000 000 / 86 400 = 1157 messages/s Pic estimé : 1157 × 3 = 3472 messages/s

Volume de messages [H1] Par jour : 100 000 000 × 300 = 30 Go Par an : 30 × 365 = 10950 Go = 10,95 To

MongoDB à 100 millions de messages [H1, H3, H4] Messages : 100 000 000 × 300 = 30 Go Index : 100 000 000 × 40 = 4 Go Total : 34 Go pour 16 Go de RAM

Cassandra en 2017 [A1, H1] Sans les trois copies : 12 × 1 To / 3 = 4 To Équivalent théorique : 4 × 10^12 / 300 ≈ 13,3 milliards de messages

Le dernier calcul donne seulement le volume. Les 4 To sont compressés, contrairement aux 300 octets supposes, même si un message prenait 150 ou 600 octets [P], messages et index dépasseraient les 16 Go de RAM.

2.c : Faut-il garder des copies ?

Il faut garder les droits d'accès à jour. Je limite donc les copies aux lectures qui peuvent attendre un peu [P].

Besoin	Idée	Risque	À prévoir
Lire les derniers messages	Cache Redis	Voir un ancien message	Vider le cache après changement
Voir les statistiques	Calcul une fois par jour	Chiffres en retard	Recalcul possible
Voir les suggestions	Cache si besoin	Suggestions dépassées	Durée d'expiration

Je ne copie pas le droit d'écrire. Quelqu'un qui quitte un serveur ne doit plus pouvoir y

envoyer de message.

3. Pourquoi les messages posaient problème

3.a : Le cas MongoDB

Discord avait un index (channel_id, created_at) [A1]. Avec mon hypothèse, les messages et l'index prennent 34 Go pour 16 Go de RAM. Même si l'index tient en mémoire, il faut parfois lire le message sur le disque. Cela explique les temps de réponse qui changent d'une lecture à l'autre.
Le cas le plus gênant est un serveur surtout vocal. Il y a peu de messages écrits, donc retrouver les 50 derniers oblige parfois à lire des messages anciens [A1].

3.b : Les quatre solutions comparées

Base	Comment on lit	Si la RAM manque	Plusieurs machines	Ce qu'on perd
PostgreSQL	Index + LIMIT 50	Accès disque	Plus dur à répartir	Répartition à gérer

MongoDB	Index salon/date	Accès disque	Sharding	Relations moins simples
Cassandra	Salon + période	Données par nœud	Prévu pour ça	Peu de jointures, plusieurs périodes
Redis	Messages en mémoire	Limite de RAM	Possible	Pas d'historique seul

Pour un serveur vocal (moins de 1 000 messages/an), le cache aide peu. Un serveur textuel privé (100 000 à 1 million/an) lit surtout du récent, pas toujours en RAM. Un grand serveur public lit beaucoup de messages récents, mais peut charger une partition [A1].
Pour 300 000 messages, je garde PostgreSQL. Redis ne sera ajouté comme cache des messages que si les tests le justifient [P].

3.c : Comment découper les messages dans Cassandra

On prend un salon qui écrit 10 millions de messages par an, à 300 octets chacun [H1], répartis sur l'année [P]. Dans les trois cas, les messages sont triés par message_id.

Option	Partition	Calcul	Taille
A	Tout le salon	10 000 000 × 300	3 000 Mo
B	Salon + 10 jours	10 000 000 × 10 / 365 × 300	82,2 Mo
C	Salon + 1 jour	10 000 000 / 365 × 300	8,2 Mo

Seule A dépasse le seuil de 100 Mo cité par Discord [A1]. Mais des petites partitions obligent parfois à lire plus de périodes.
Avec 1 000 messages/an, retrouver 50 messages couvre environ 50 × 365 / 1 000 = 18,25 jours. On lit environ 2 à 3 partitions avec B, contre 19 à 20 avec C [P]. Je retiens donc 10 jours comme compromis pour cette comparaison. Il faut aussi ignorer les périodes sans message. message_id est mieux que la date pour le tri, parce que deux messages peuvent avoir la même date [A1].

3.d : Les incidents à éviter

Incident	Ce qui s'est passé	Ce qu'on pouvait prévoir	Solution envisagée [P]

Champs NULL [A1]	Environ 12 tombstones par message	Trop de champs vides	Ne pas écrire les champs absents
Millions de suppressions [A1]	Lecture ralentie jusqu'à 20 s	Trop de tombstones	Suivre les suppressions et éviter les périodes vides
Partition surchargée [A2]	Trop de requêtes sur un nœud	Salons avec activité très différente	Surveiller la charge par partition
Pauses de Cassandra [A2]	La JVM bloque parfois un nœud	Gestion de la mémoire	Surveiller les pauses et prévoir une reprise

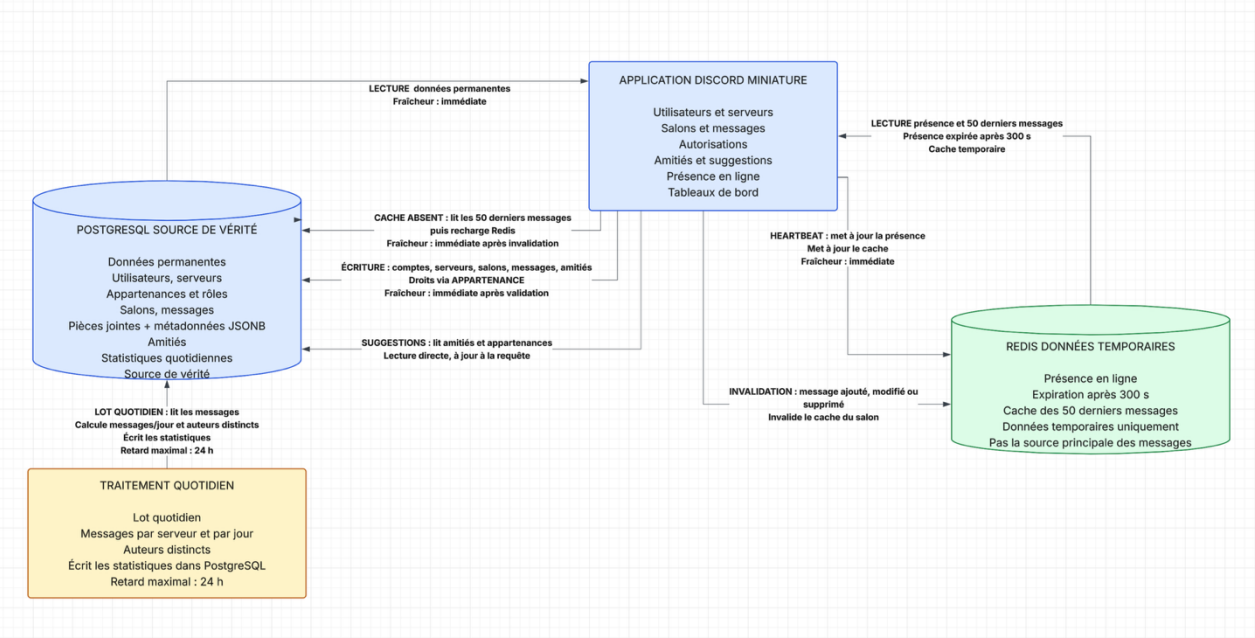
4. L'architecture que je propose

4.a : Où je mets chaque donnée

Besoin	Base	Type	Règle ou chiffre	Source de vérité	Arrivée et limite
Comptes	PostgreSQL	Relationnel	E-mail unique	PostgreSQL	Inscription, doublon refusé
Serveurs et membres	PostgreSQL	Relationnel	Un propriétaire	PostgreSQL	Ajout, transfert à contrôler
Salons	PostgreSQL	Relationnel	Nom unique	PostgreSQL	Création, pas de doublon
Messages et fichiers	PostgreSQL	Relationnel + JSONB	Auteur et salon texte	PostgreSQL	Envoi, contrôle des champs
Droit d'écrire	PostgreSQL	Relationnel	Auteur membre	PostgreSQL	Vérification à chaque envoi
Amitiés	PostgreSQL	Relationnel	Une paire unique	PostgreSQL	Ajout ou retrait
Présences	Redis	Clé-valeur	5 minutes	Redis	Signe de vie, perdu si panne
Suggestions	PostgreSQL	Relationnel	Amis en commun	PostgreSQL	Calcul, parfois plus lent
Statistiques	PostgreSQL	Table calculée	Une fois par jour	Messages SQL	Calcul nocturne,

					retard d'un jour
50 derniers messages	PostgreSQL (+ Redis si utile)	SQL + cache	Lecture N° 1	PostgreSQL	Lecture, cache parfois en retard

4.b : Schéma des échanges



L'application écrit les comptes, serveurs, salons et messages dans PostgreSQL. La présence va dans Redis avec 300 secondes d'expiration. Les statistiques sont recalculées chaque nuit. Si j'active le cache des messages, je le vide après une modification [P].

4.c : Mes trois décisions

1. Garder PostgreSQL pour les données. J'ai regardé MongoDB et Cassandra. MongoDB demanderait plus de vérifications dans l'application pour les relations. Cassandra sert surtout quand on répartit beaucoup de messages. Pour nos 300 000 messages, PostgreSQL permet de gérer les liens et les contraintes avec une seule base. Par contre, si la lecture des 50 messages passe souvent au-dessus de 80 ms au p95 malgré les index, ou si les envois ralentissent, faudra revoir ce choix [P].

2. Utiliser Redis pour la présence. J'aurais pu stocker les personnes en ligne dans PostgreSQL, mais il faudrait nettoyer les présences après cinq minutes. Redis fait expirer les clés tout seul. Je peux aussi m'en servir pour les derniers messages, mais pas comme seule base d'historique. Le défaut, c'est qu'il faut gérer deux outils. Si le cache n'apporte pas de gain, je l'enleve pour les messages [P].

3. Ne pas tout mettre en cache. J'ai pensé à copier aussi les droits d'écriture, mais un ancien droit pourrait laisser quelqu'un écrire après son départ d'un serveur. Je garde donc les appartenances dans PostgreSQL. Les statistiques, elles, peuvent attendre le lendemain. Si elles ont plus de 24 h de retard ou si les suggestions dépassent souvent 200 ms au p95, il faudra revoir les calculs [P].

4.d. Le risque que je vois

Un message est supprimé de PostgreSQL, mais il reste dans Redis. Quelqu'un l'ouvre et le voit encore. Pour détecter ça, je prévois de noter les erreurs du cache et de comparer de temps en temps quelques salons aux données SQL. En cas d'écart, on vide la copie Redis. Si Redis tombe en panne, on lit les messages dans PostgreSQL [P].

4.e : La version qu'on peut faire à deux

Le sujet limite l'équipe à deux personnes et le budget à deux moteurs. Je garde donc PostgreSQL et Redis. Les suggestions restent en SQL, comme les statistiques. Je n'ajoute pas de base de graphes pour 15 000 amitiés [P].

Ajout possible [P]	Ce qui me ferait regarder	Avant de changer
Cassandra	Plus de 10 millions de messages, p95 > 80 ms ou erreurs > 1 %	Tester les index et le cache
Base de graphes	Plus d'un million d'amitiés, p95 > 200 ms ou erreurs > 1 %	Tester les requêtes SQL

C’est des seuils proposés pas les performances mesurés

4.f. Hypothèses pour le jour 2

Je pense que Redis sera 3 fois plus rapide que PostgreSQL pour lire 50 messages, car ils sont déjà en cache. sinon le cache n'est peut-être pas utile

Je pense que PostgreSQL sera 1,2 fois plus rapide que Redis avec un cache vide, car il évite une vérification. sinon le cache vide ralentit peu

Je pense que Redis sera 5 fois plus rapide que PostgreSQL pour voir qui est en ligne, grâce à l'accès par clé. sinon PostgreSQL peut suffire

Je pense que Redis sera 2 fois plus rapide que PostgreSQL pour vérifier un membre déjà en cache. sinon je garde PostgreSQL

Je pense que Redis sera 2 fois plus rapide que PostgreSQL pour trouver des amis d'amis déjà en cache. sinon je garde les requêtes SQL

5. Ce que je pense avant les test

Test	Mon idée [P]	Si je me trompe
Lire 50 messages, cache plein	Redis environ 3 fois plus rapide que PostgreSQL, car tout est en mémoire	Garder la lecture SQL si le gain est faible
Lire 50 messages, cache vide	PostgreSQL seul environ 1,2 fois plus rapide qu'un passage Redis puis SQL, car il y a un accès de plus	Le coût du cache vide sera plus faible que prévu
Lire une présence	Redis environ 5 fois plus rapide que PostgreSQL avec sa lecture par clé	Redis peut quand même servir pour l'expiration
Vérifier une appartenance	Copie Redis environ 2 fois plus rapide qu'une vérification SQL, sans jointure	Garder SQL pour les droits, même si Redis gagne
Trouver des amis d'amis	Redis environ 2 fois plus rapide avec des listes déjà chargées	Garder les jointures PostgreSQL

Les copies Redis utilisées pour comparer les tests 4 et 5 ne font pas partie de l'architecture retenue. Je n'ai pas encore fait ces mesures

Annexe. MPD proposé en SQL
Voici le script prévu pour PostgreSQL. Il sera à exécuter et à verifier :

```
CREATE TABLE utilisateur (
  id BIGINT PRIMARY KEY,
  pseudo VARCHAR(80) NOT NULL,
  email VARCHAR(254) NOT NULL UNIQUE
);
CREATE TABLE serveur (
  id BIGINT PRIMARY KEY,
  nom VARCHAR(120) NOT NULL
);
CREATE TABLE appartenance (
  serveur_id BIGINT NOT NULL REFERENCES serveur(id),
  utilisateur_id BIGINT NOT NULL REFERENCES utilisateur(id),
  role VARCHAR(12) NOT NULL
    CHECK (role IN ('proprietaire', 'moderateur', 'membre')),
  PRIMARY KEY (serveur_id, utilisateur_id)
);
CREATE UNIQUE INDEX un_proprietaire_par_serveur
  ON appartenance(serveur_id) WHERE role = 'proprietaire';
```

```
CREATE INDEX idx_appartenance_utilisateur
  ON appartenance(utilisateur_id, serveur_id);
CREATE TABLE salon (
  id BIGINT PRIMARY KEY,
  serveur_id BIGINT NOT NULL REFERENCES serveur(id),
  nom VARCHAR(120) NOT NULL,
  type VARCHAR(8) NOT NULL CHECK (type IN ('textuel', 'vocal')),
  UNIQUE (serveur_id, nom)
);
CREATE TABLE message (
  id BIGINT PRIMARY KEY,
  salon_id BIGINT NOT NULL REFERENCES salon(id),
  auteur_id BIGINT NOT NULL REFERENCES utilisateur(id),
  contenu TEXT NOT NULL,
  envoye_le TIMESTAMPTZ NOT NULL,
  modifie_le TIMESTAMPTZ,
  CHECK (modifie_le IS NULL OR modifie_le >= envoye_le)
);
CREATE INDEX idx_message_salon_id_desc ON message(salon_id, id
DESC);
CREATE INDEX idx_message_salon_date ON message(salon_id, envoye_le);
CREATE TABLE piece_jointe (
  id BIGINT PRIMARY KEY,
  message_id BIGINT NOT NULL UNIQUE REFERENCES message(id),
  type VARCHAR(7) NOT NULL CHECK (type IN ('image','fichier','lien')),
  reference TEXT NOT NULL,
  metadonnees JSONB NOT NULL DEFAULT '{}'::jsonb
);
CREATE TABLE amitie (
  utilisateur_min BIGINT NOT NULL REFERENCES utilisateur(id),
  utilisateur_max BIGINT NOT NULL REFERENCES utilisateur(id),
  PRIMARY KEY (utilisateur_min, utilisateur_max),
  CHECK (utilisateur_min < utilisateur_max)
);
CREATE INDEX idx_amitie_second ON amitie(utilisateur_max, utilisateur_min);
CREATE TABLE stat_jour (
  serveur_id BIGINT NOT NULL REFERENCES serveur(id),
  jour DATE NOT NULL,
  nb_messages INTEGER NOT NULL CHECK (nb_messages >= 0),
  nb_auteurs_distincts INTEGER NOT NULL CHECK (nb_auteurs_distincts >=
0),
  PRIMARY KEY (serveur_id, jour),
  CHECK (nb_auteurs_distincts <= nb_messages)
);
```

Le code de l'application devra aussi gérer le transfert de propriétaire et refuser les messages venant d'un non-membre ou destinés à un salon vocal. Ces deux points ne sont pas entièrement garantis par le script ci-dessus.

Repères : [A1] article Discord de 2017, [A2] article de 2023, [H1] à [H5] hypothèses données dans le sujet, [P] proposition personnelle